

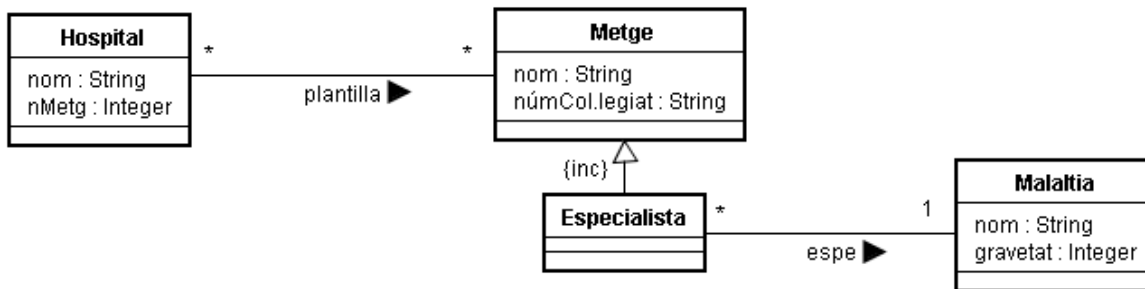
ARQUITECTURA DEL SOFTWARE

Unit 3.4: Data Layer Design

Exercicis Resolts

Exercici 2

Es vol implementar el tros de model conceptual següent:



Restriccions d'integritat de clau: Hospital -> nom, Metge-> nom, Malaltia-> nom

Ens interessen les dues operacions següents:

```
context CapaDeDomini::metgePlega(nomH: String, nomM: String)
pre el Metge nomM existeix i no és Especialista
pre l'Hospital nomH existeix
exc metge-no-hi-treballa: el Metge nomM no treballa a l'Hospital nomH
post enregistra que el Metge nomM deixa de treballar a l'Hospital nomH
post si el Metge nomM només és a la plantilla de l'hospital nomH, esborra el Metge
post decrementa el nombre de metges en plantilla de l'hospital, nMetg
```

```
context CapaDeDomini::malaltiaMésGreu(): String
exc no-hi-ha-malalties: no hi ha cap Malaltia enregistrada en el sistema
post retorna el nom de la Malaltia que té gravetat més gran
```

Es demana:

- Feu l'esquema de la base de dades. Les úniques responsabilitats que es poden assignar a l'esquema de la base de dades són les de clau (la clau és el nom en totes les classes). Useu l'estratègia Class Table Inheritance per tractar la jerarquia d'herència.
- Feu el disseny de les operacions usant patrons Transaction Script amb Pasarel.la Fila (sense operacions de consulta en la Capa de Gestió de Dades, és a dir, que les operacions de consulta retornin pasarel.les del mateix tipus que el cercador).
- Repetiu *malaltiaMésGreu* però ara permetent operacions de consulta en la Capa de Gestió de Dades.
- Feu el diagrama de classes de disseny de la capa de gestió de dades.
- Feu el disseny de l'operació *metgePlega* usant patrons Transaction Script amb Active Record (sense operacions de consulta en la Capa de Gestió de Dades).

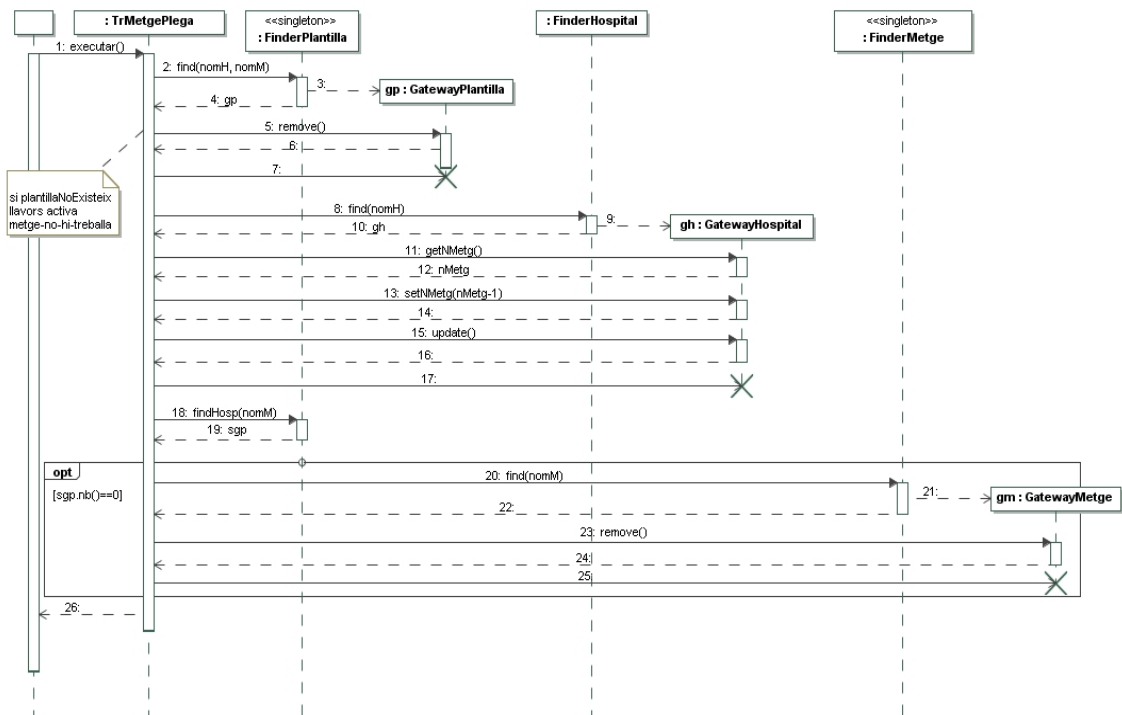
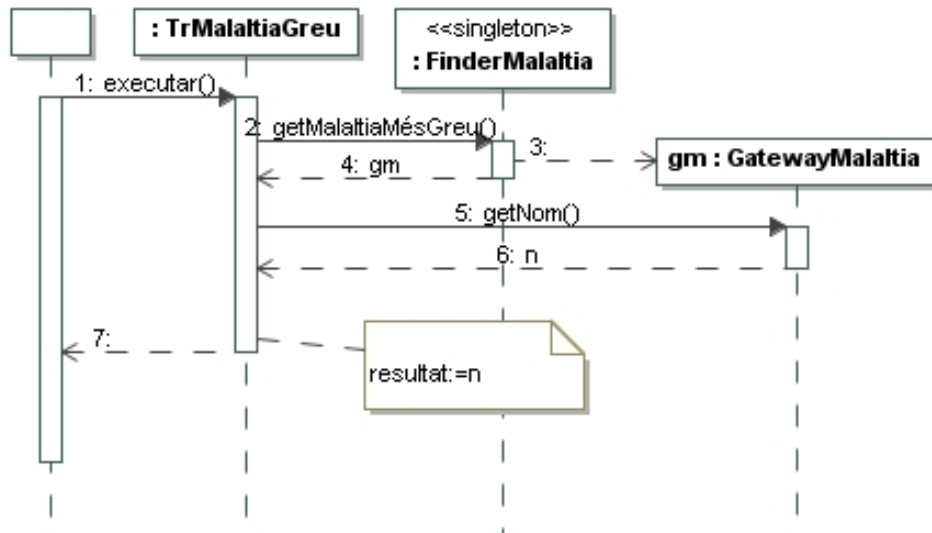
Solució

(a)

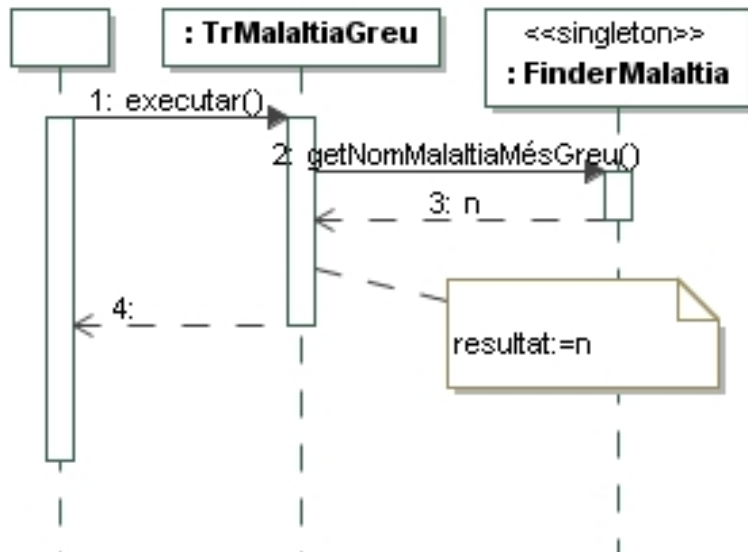
Hospital (nom, nMetge)

Metge (nom, numCol.legiat)
 Plantilla (nomHosp, nomMetge)
 {nomHosp} referencia Hospital
 {nomMetge} referencia Metge
 Especialista (nomEsp, nomMal)
 {nomEsp} referencia Metge
 {nomMal} referencia Malaltia
 Malaltia (nom, gravetat)

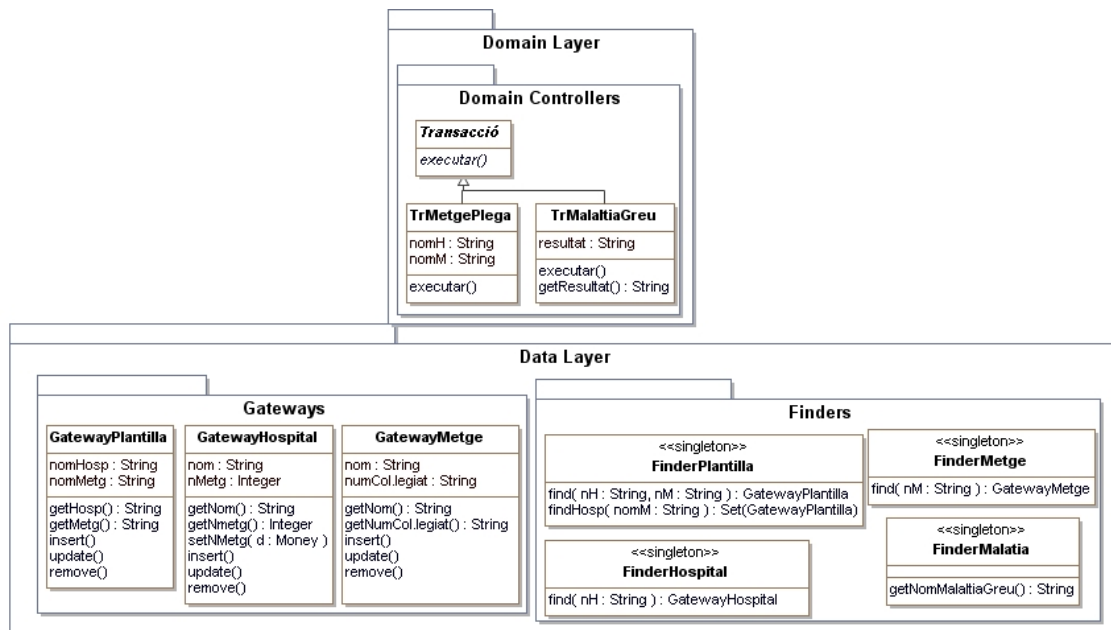
(b)



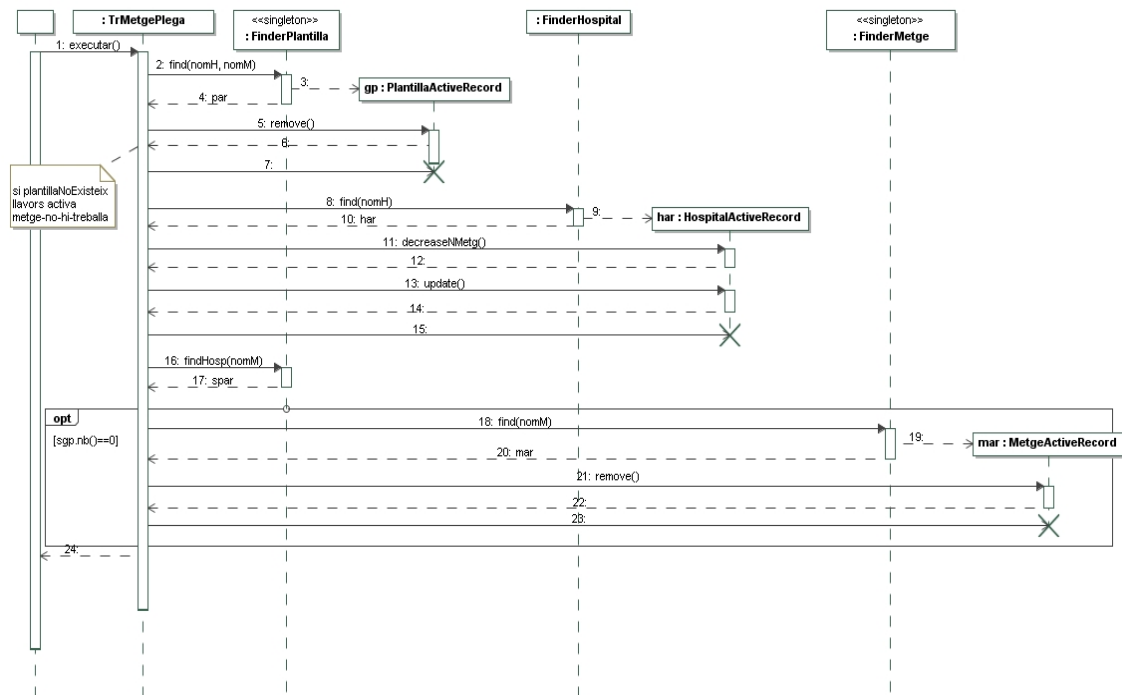
(c)



(d)

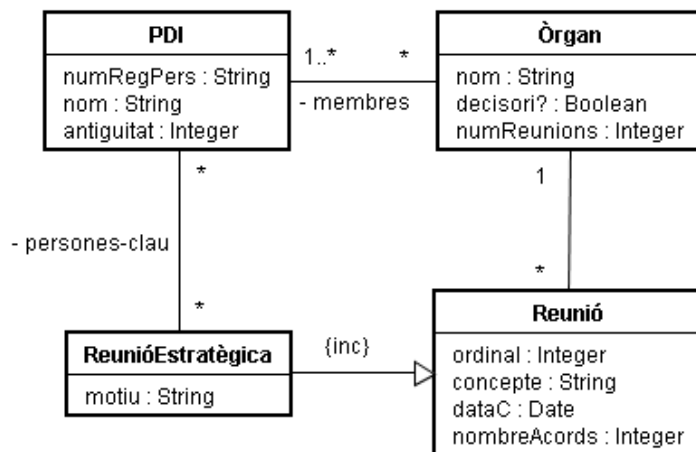


(e)



Exercici 3

Es disposa d'una variant/simplificació del conegut problema de les reunions de la UPC. El diagrama de classes mostra que els Òrgans poden ser decisoris i enregistren el nombre de reunions fetes, que les Reunions enregistren el nombre d'acords que s'hi van prendre, i que s'enregistren els PDI que són persones clau en Reunions Estratègiques.



Restriccions d'integritat de clau (la resta no són importants per al problema):

PDI → numRegPers

Òrgan → nom

Reunió → Òrgan::nom+
ordinal+concepte

Ens interessen les dues operacions següents:

context CapaDeDomini::novaReunióEstratègica(nomO: String, concR: String, ordR: Int, dataR: Date, nacR: Int, motR: String, keyP: Set(String))

pre PDIs-existeixen: tots els números de registre personal del conjunt keyP són de PDI existents al sistema

exc òrgan-no-existeix: l'òrgan nomO no existeix

exc reunió-existeix: la reunió nomO+concR+ordR existeix

post es dóna d'alta una reunió estratègica per a l'òrgan nomO amb concepte concR i ordinal ordR, amb motiu motR, data dataR, nombre d'acords nacR i persones clau keyP
post s'incrementa el nombre de reunions enregistrat per a l'òrgan nomO

context CapaDeDomini::llistat(nrpP: String): Set(nomO: String+Set(concR: String+ordR: Int))
pre PDI-existeix: existeix el PDI nrpP
post per a cada òrgan decisor de la UPC retorna el seu nom amb el conjunt de reunions estratègiques (concepte i ordinal) en les què nrpP era persona clau i no s'hi ha pres cap acord

(a) Feu l'esquema de la base de dades. Useu l'estratègia Concrete Table Inheritance per tractar la jerarquia d'herència. Indiqueu les claus primàries i foranes.

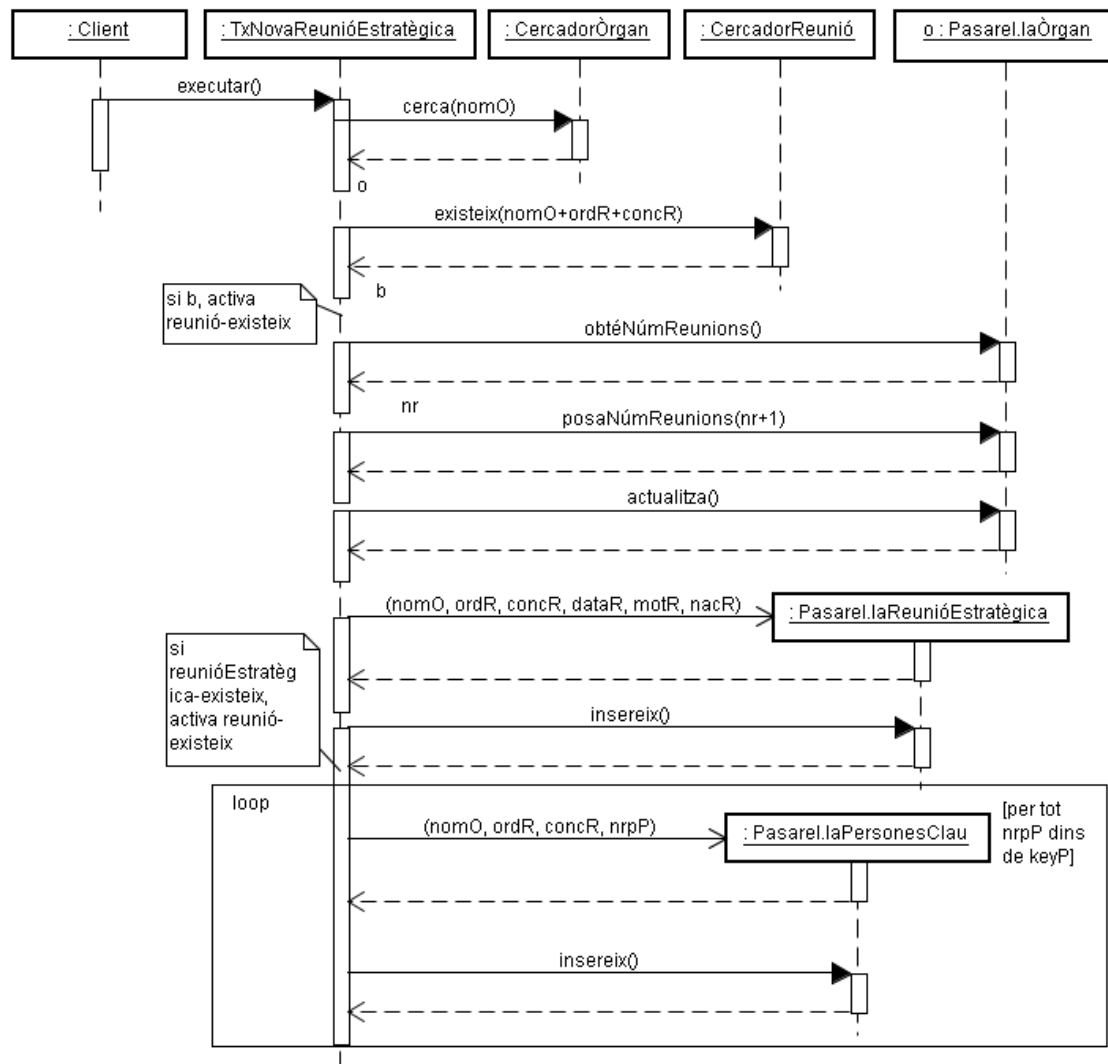
(b) Feu el disseny de les operacions usant patrons Transaction Script amb Pasarel·la Fila (en la seva versió bàsica, és a dir, sense operacions de consulta arbitràries en la Capa de Gestió de Dades). Les úniques responsabilitats que es poden assignar a l'esquema de la base de dades són les de clau. No cal destruir les pasarel·les. Declareu el contracte de les operacions de consulta que necessiteu a part de les típiques que tenen sempre els cercadors.

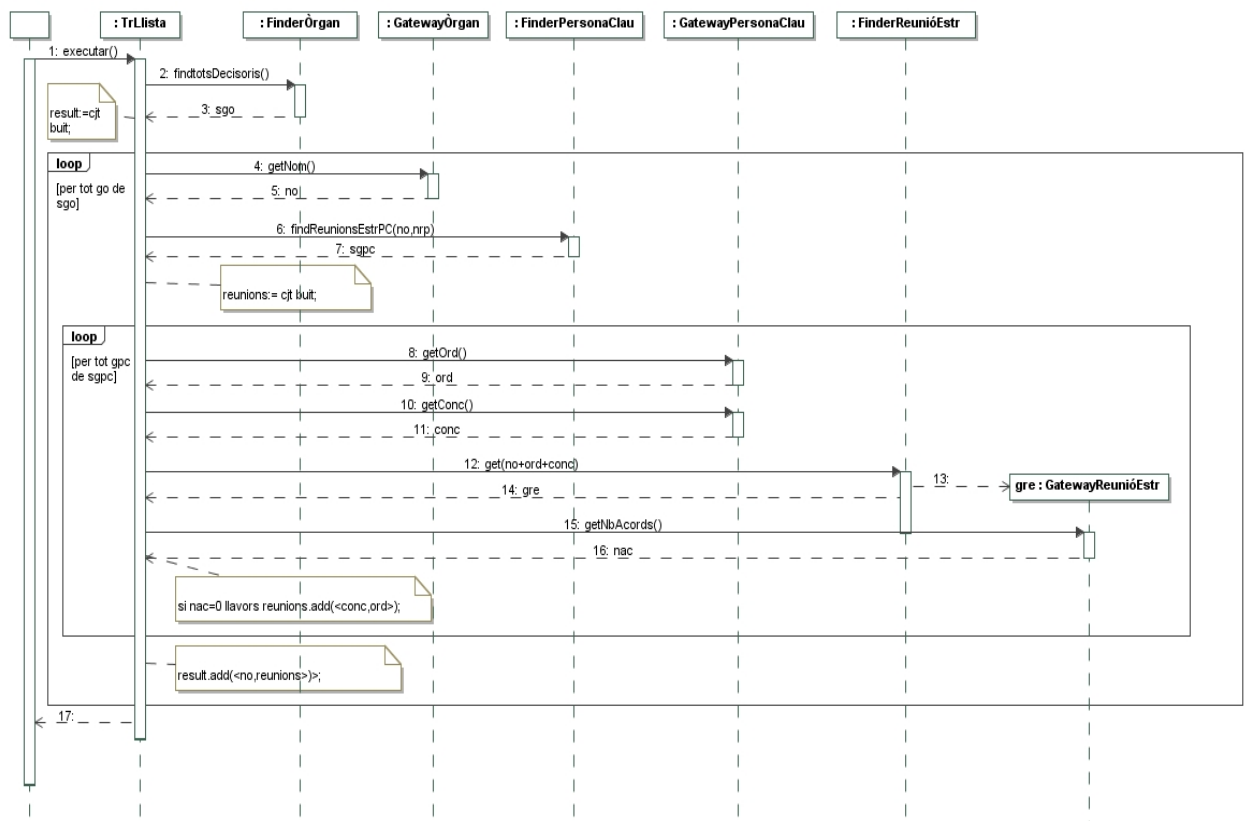
Solució

(a)

Òrgan(nom, decisor?, numReunions)
PDI(númRegPers, nom, antiguitat)
membres(númRegPers, nom)
 {númRegPers} referencia PDI
 {nom} referencia Òrgan
persones-clau(númRegPers, nom, ordinal, concepte)
 {númRegPers} referencia PDI
 {nom, ordinal, concepte} referencia ReunióEstratègia
Reunió(nom, ordinal, concepte, dataC, nombreAcords)
 {nom} referencia Òrgan
ReunióEstratègia(nom, ordinal, concepte, dataC, nombreAcords, motiu)
 {nom} referencia Òrgan

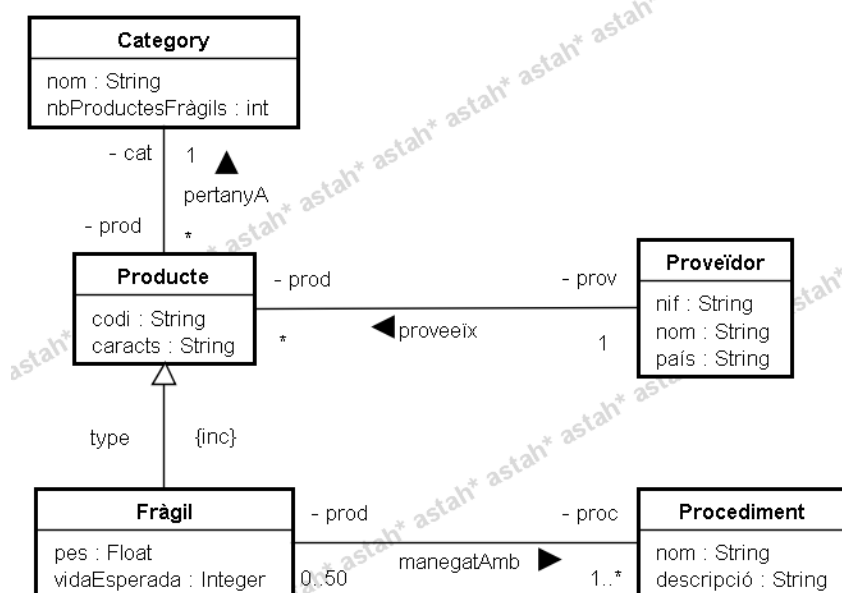
(b)





Exercici 5

Una companyia d'àmbit mundial manega un catàleg de Productes que pertanyen a una categoria i que són manufacturats per Proveïdors. Alguns productes són fràgils i tenen assignats procediments de conservació per assegurar que s'entreguen correctament.



Restriccions d'integritat:

IC1: (Categoria → nom), (Producte → codi), (Procediment → nom), (Proveïdor → nif)

Considereu el contracte següent que està dins del cas d'ús *NouProducteFràgil*:

context CapaDomini::nouProducteFràgil(codP: String, carP: String, pesP: Float, vidaEsp: Integer, nifP: String, categP: String): Set(nameP: String)

exc producte-existeix: el producte amb codi = codP ja existeix

exc categoria-no-existeix: la categoria amb nom = categP no existeix

pre proveïdor-existeix: el proveïdor amb nif = nifP existeix

post crea un nou producte fràgil amb els atributs donats i lligat a la categoria i proveïdor donats

post incrementa el número de productes fràgils de la categoria

post retorna el conjunt de noms de procediments que encara no tenen 50 productes assignats

Us demanem:

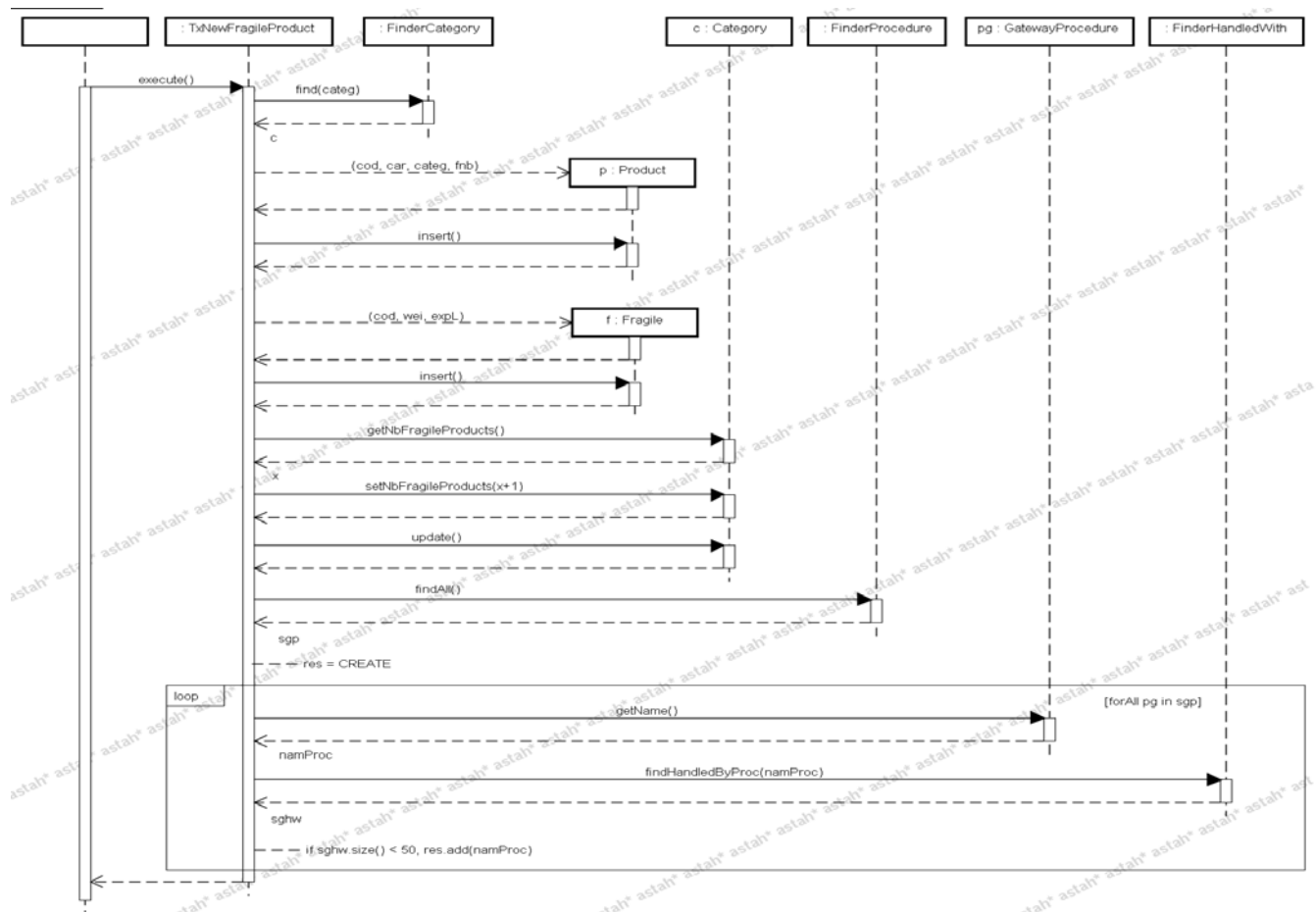
- a) Dissenyeu l'esquema relacional corresponent al diagrama de classes usant Class Table Inheritance per transformar la jerarquia d'herència.
- b) Dissenyeu l'operació anterior usant els patrons Transaction Script i Pasarel·la Fila en la seva forma més bàsica (i.e., sense operacions arbitràries de consulta). No assigneu responsabilitats a l'esquema relacional a banda de les de clau primària. No cal destruir les pasarel·les. Escriviu el contracte de les operacions cercadores diferents de l'operació de cerca per clau primària.
- c) Repetiu l'apartat anterior però ara usant operacions arbitràries de consulta.
- d) Imagineu que preferim usar Concrete Table Inheritance per a la construcció de l'esquema relacional. Escriviu la part de l'esquema que és diferent respecte de l'apartat (a). Quin és l'impacte sobre el diagrama de seqüència anterior?

Solució

a)

Category(name, nbFragileProducts), primary key: name
Supplier(fiscalNb, name, country), primary key: fiscalNb
Procedure(name, description), primary key: name
HandledWith(prod, proc), primary key: prod+proc
foreign keys: prod of Product, proc of Procedure
Product(code, caracts, cat, sup), primary key: code
foreign keys: cat of Category, sup of Supplier
Fragile(code, weight, expectedLife), primary key: code
foreign keys: code of Product

b)



context FinderProcedure::findAll(): Set(GatewayProcedure)

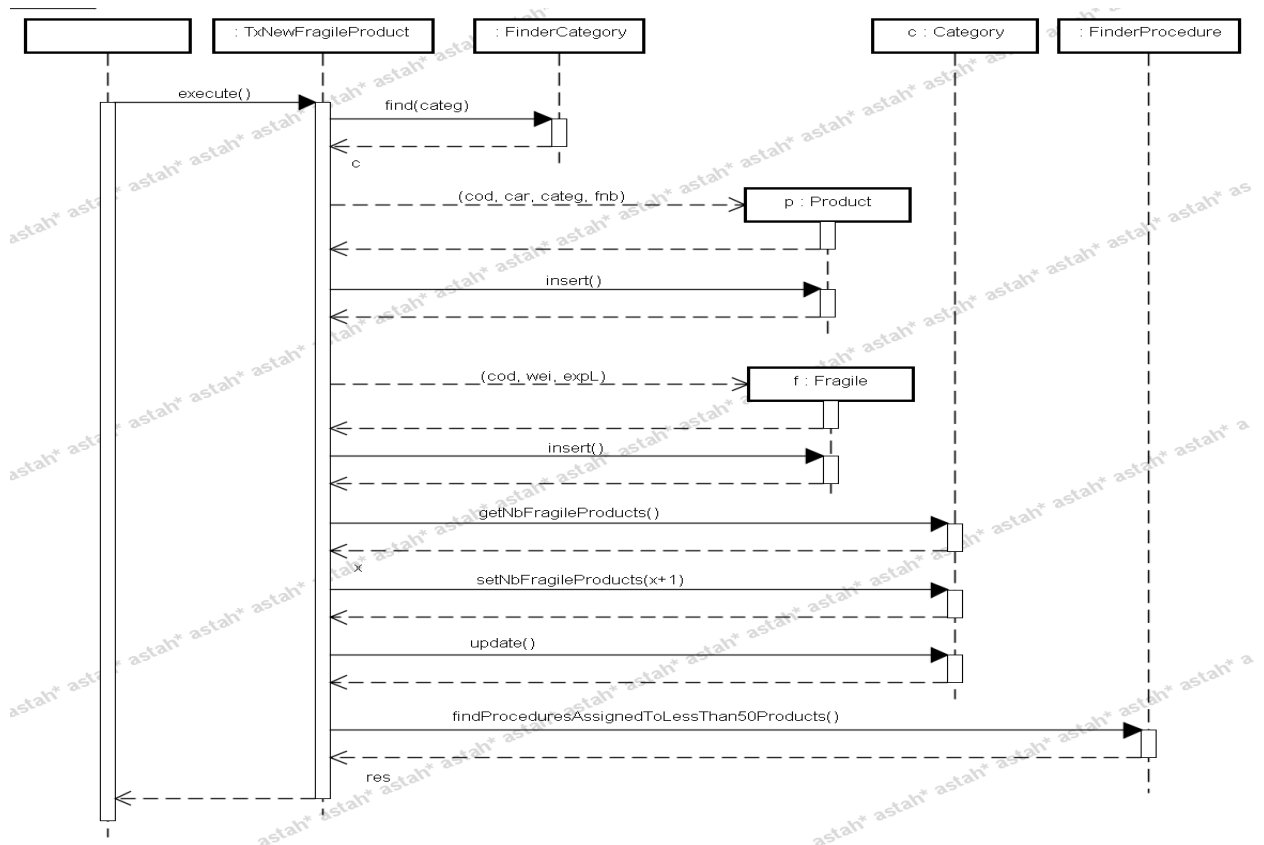
post returns the set of gateways to all the procedures stored in the system

context FinderHandledWith::findHandledByProc(namP: String):

Set(GatewayHandledWith)

post returns the set of gateways to HandledWith that imply the given procedure *namPk*

c)

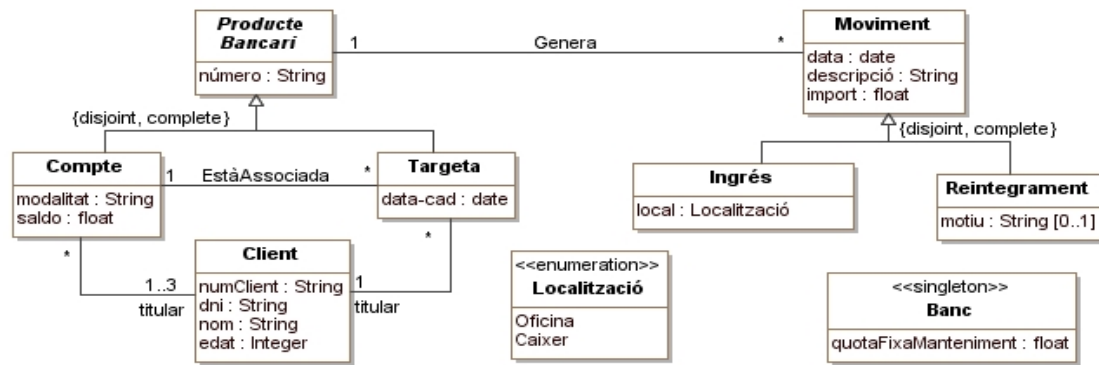


context FinderProcedure::findProceduresAssignedToLessThan50Products():
Set(nameP: String)
post returns the set of names of procedures that have less than 50 fragile products assigned

d)
Fragile(code, caracts, cat, sup, weight, expectedLife), primary key: code
foreign keys: cat of Category, sup of Supplier
No cal crear el Producte. Només cal crear Fràgil. Ara, el productes i els fràgils són disjunts.
L'excepció de que el producte fràgil existeix s'haurà de convertir a producte existeix

Exercici 7

El departament que s'encarrega de gestionar els productes bancaris d'un banc vol modernitzar un dels seus sistemes software. S'ha adonat que part de la informació que manté el sistema software (representada en l'esquema conceptual) és també proporcionada per serveis software que proporcionen altres departaments del banc. Amb l'objectiu de reduir el manteniment d'aquesta informació redundant ens han demanat redissenyar el sistema.



R.I. Textuals:

RI1. Claus: (Client, dni); (Producte Bancari, número)

RI2. Un client també s'identifica per numClient

RI3. Tots els integers i floats excepte el saldo han de ser positius.

RI4. El titular d'una targeta ha de ser un dels titulars del compte associat.

RI5. Tots els reintegraments amb un import superior a 1000 euros han de tenir el motiu del reintegrament.

RI6. El saldo d'un compte bancari és el sumatori de tots els imports dels ingressos del compte i de totes les targetes associades menys el sumatori de tots els imports dels reintegraments del compte i de totes les targetes associades.

RI7. Altres restriccions no rellevants pel problema

Considerant el diagrama de classes i restriccions d'integritat, es demana:

- Disseny de la base de dades usant Class Table Inheritance per a la jerarquia de Productes Bancaris i Concrete Table Inheritance per a la jerarquia de Moviments. Useu restriccions de clau i altres restriccions de columna i de taula sempre que es pugui (**No es poden usar triggers**).
- [2.5 punts] Disseny de l'operació següent usant el **patró Transaction Script amb Row Data Gateway** (Pasarel·la Fila) (**sense operacions arbitràries de consulta**). Mostreu clarament en quin moment es llancen les excepcions indicades en l'operació.

context CapaDomini:: AltaTargeta(numTargeta: String, dataCad: date, numCompte: String, dniCl: String, descrIngrés: String, importIngrés: float, localIngrés: Localització): Set(TupleType(nomClient: String, saldoGlobalComptes: float))

pre clientExisteix: el client amb dni *dniCl* existeix.

pre compteExisteix: el compte amb número *numCompte* existeix.

exc producteBancariExisteix: existeix un producte bancari amb número *numTargeta*.

exc clientNoTitular: el client *dniCl* no és titular del compte *numCompte*.

exc titularCompteSaldoGlobalNegatiu: algun dels titulars del compte *numCompte* té un saldo global dels comptes del banc negatiu. El saldo global dels comptes d'un client és la suma del saldo de tots els comptes dels que el client és titular.

post targetaCreada: s'ha creat la targeta *numTargeta* i data de caducitat *dataCad*, així com l'associació de la targeta amb el compte *numCompte*, i l'associació de la targeta amb el client amb *dniCl*.

post ingrésCreat: s'ha creat un moviment de tipus ingrés associat a la targeta creada, amb data actual (podeu invocar l'operació *getDataActual():date*), descripció *descrIngrés*, import *importIngrés*, i localització *localIngrés*.

post saldoModificat: s'ha incrementat el saldo del compte associat a la targeta amb la quantitat corresponent a *importIngrés*.

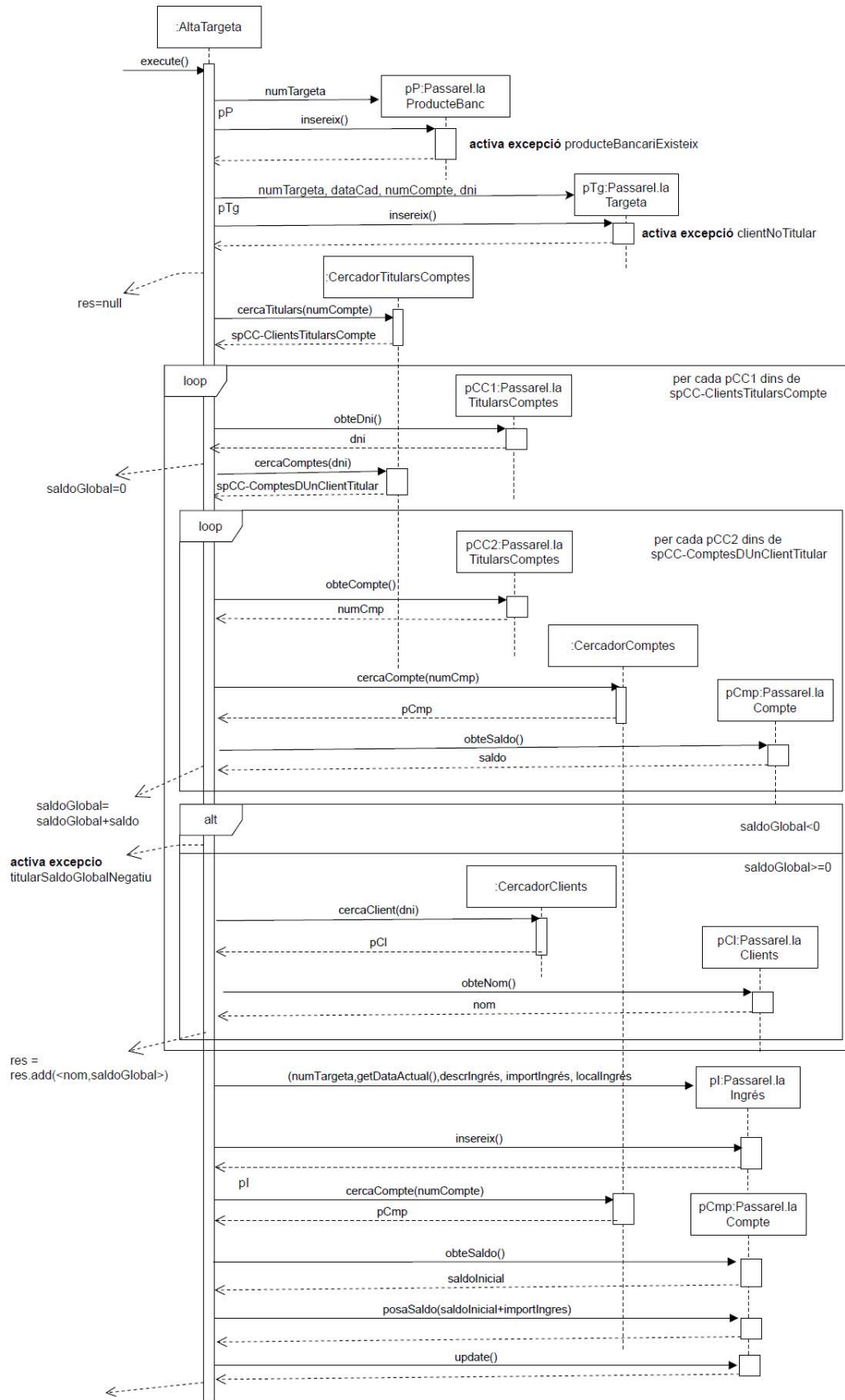
post clientsCoTitulars: es retorna per cada client que és titular del compte associat a la targeta *numTargeta*, el nom del client i el saldo global dels comptes del client en el banc.

Solució

a)

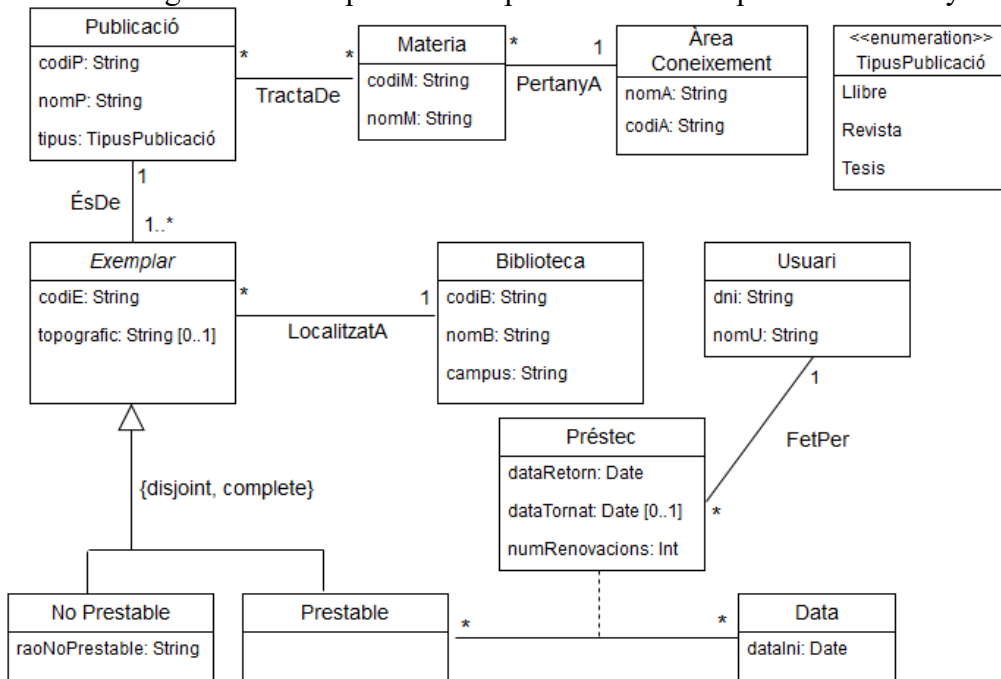
```
clients (dni, numClient, nom, edat)
    {numClient} unique, not null /* RI2 */
    check(edat>0) /* RI3 */
productesBancarís (número)
comptes (numCompte, modalitat, saldo)
    {numCompte} referencia productesBancarís
titularsComptes(dni,numCompte)
    {dni} referencia clients
    {numCompte} referencia comptes
targetes (numTargeta, data-cad, numCompte,dni)
    {numTargeta} referencia productesBancarís
    {dni,numCompte} referencia titularsComptes /* RI4, sino es posa calen dues
    FKs a clients i comptes*/
    dni , numCompte NOT NULL
ingressos (idIngrés, número, data, descripció,import,local)
    {número} referencia productesBancarís NOT NULL
    check(import>0) /* RI3*/
    check(local in ('Oficina','Caixer'), local Not Null /* enumerat */
reintegraments (idReintegrament, número, data, descripció, import, motiu)
    {número} referencia productesBancarís NOT NULL
    check(import>0) /* RI3 */
    check(import<=1000 or motiu is not null) /* RI5 */
banc (quotaFixaManteniment)
/* RI1, són les PKs de les taules clients, productesBancarís, comptes, targetes */
RI6 no és pot implementar sense triggers.
Cal afegir una restricció després de la traducció per garantir que un compte no té més
de 3 titulars i 1 mínim.
```

b)



Exercici 8

Una universitat ens contacta per que li dissenyem un sistema software per gestionar les seves biblioteques. El sistema ha de gestionar la informació de les publicacions i exemplars que hi ha a les diferents biblioteques i els seus préstecs. A continuació disposeu d'un fragment de l'esquema conceptual del sistema que es vol dissenyar.



R.I. Textuals:

- Claus: (Biblioteca, codiB), (Publicació, codiP), (Usuari, dni), (Matèria, codiM), (Àrea Coneixement, codiA)
- Una publicació no pot tenir més d'un exemplar amb el mateix codiE
- Totes les matèries a les que pertany una publicació són de la mateixa àrea de coneixement.
- Un exemplar només pot tenir 1 préstec en curs (sense dataTornat).
- La data de retorn ha de ser igual a la data en la que es fa el préstec més 10 dies per cada renovació que s'ha fet del préstec.
- La data de tornat ha de ser posterior a la data en que s'ha fet el préstec.
- El número de renovacions d'un préstec és un atribut amb un valor més gran o igual que zero.
- Altres restriccions no rellevants pel problema.

context CapaDomini:: AltaPublicacio(codiP: String, nomP: String, tipus: {Llibre, Revista, Tesis}, setMateries: Set(codiM: String), setExemplars: Set(exemplars: TupleType(codiE: String, situacio:{Prestable,NoPrestable}, raoNoPrestable: String [0..1], codiB: String))) : Set(TupleType(codiB: String, nomB: String, quantitat: int))

pre materiesExisteixen: les matèries del setMateries existeixen.

pre bibliotequesExisteixen: les biblioteques del setExemplars existeixen.

pre noPrestable: raoNoPrestable té valor per exemplars en situació NoPrestable.

pre noRepetits: no hi ha exemplars repetits en el setExemplars passat com a paràmetre.

exc noMateixaArea: les matèries del setMateries no són de la mateixa àrea de coneixement.

exc publicacioExisteix: ja existeix una publicació amb codiP.

post publicacioCreat: s'ha creat la publicació i els exemplars de la publicació, així com també les associacions de la publicació amb les matèries i dels exemplars amb les biblioteques. Cada exemplar s'ha creat en la situació indicada als paràmetres d'entrada, i l'atribut tipogràfic a nul.

post *ExemplarsEnPréstec*: es retorna per cada biblioteca del sistema, el codi, el nom de la biblioteca, i la quantitat d'exemplars que té en préstec en curs.

- a) Esquema de la base de dades usant Class Table Inheritance per a la jerarquia. Només es poden usar restriccions de clau i altres restriccions de columna i de taula (**No es poden usar triggers**).
- b) Feu el disseny de l'operació *AltaPublicacio* usant **el patró Transaction Script amb Row Data Gateway** (Pasarel.la Fila) (**sense operacions arbitràries de consulta**). Mostreu clarament en quin moment es llancen les excepcions indicades en l'operació.
- c) Expliqueu en cas de poder definir operacions arbitràries de consulta quins canvis introduiríeu en el disseny de la transacció anterior. Indicant quines són les operacions de consulta que s'usaran amb paràmetres d'entrada i sortida d'aquestes operacions.

Solució

a)

usuaris(dni, nomU)
nomU not null

biblioteques(codiB, nomB, campus)
nomB not null
campus not null

publicacions(codiP, nomP, tipus)
nomP not null
tipus not null
check (tipus in ('Llibre','Revista', 'Tesis'))

exemplars(codiP, codiE, topografic, codiB)
{codiP} referencia publicacions
{codiB} referencia biblioteques
codiB not null

exNoPrestable(nomP, codiE, raoNoPrestable)
{nomP,codiE} referencia exemplars
raoNoPrestable not null

exPrestable(codiP, codiE)
{codiP,codiE} referencia exemplars

prestecs(codiP, codiE, dataIni, numRenov, dataTornat, dataRetorn, dni)
{codiP,codiE} referencia exPrestable
{dni} referencia usuaris
numRenov not null
dataRetorn not null
check(numRenov>0)
check(dataTornat>dataIni)
check(dataRetorn>dataIni)

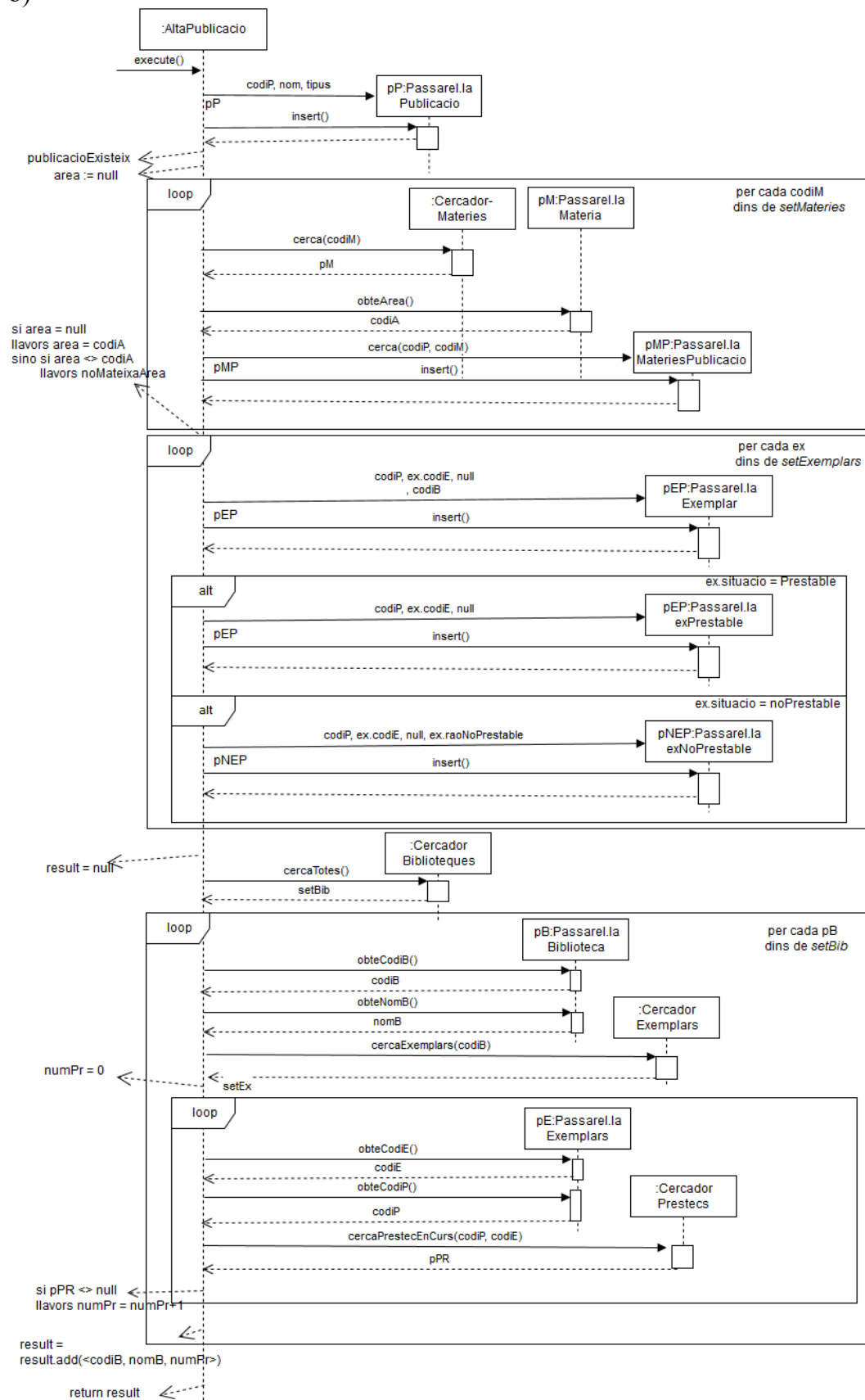
areaConeixement(codiA, nomA)
nomA not null

materies(codiM, nomM, codiA)
{codiA} referencia areaConeixement
nomM not null
codiA not null

materiesPublicacions(codiP, codiM)

{codiP} referencia publicacions
 {codiM} referencia matèries

b)



c)

Hi ha dues possibles millores:

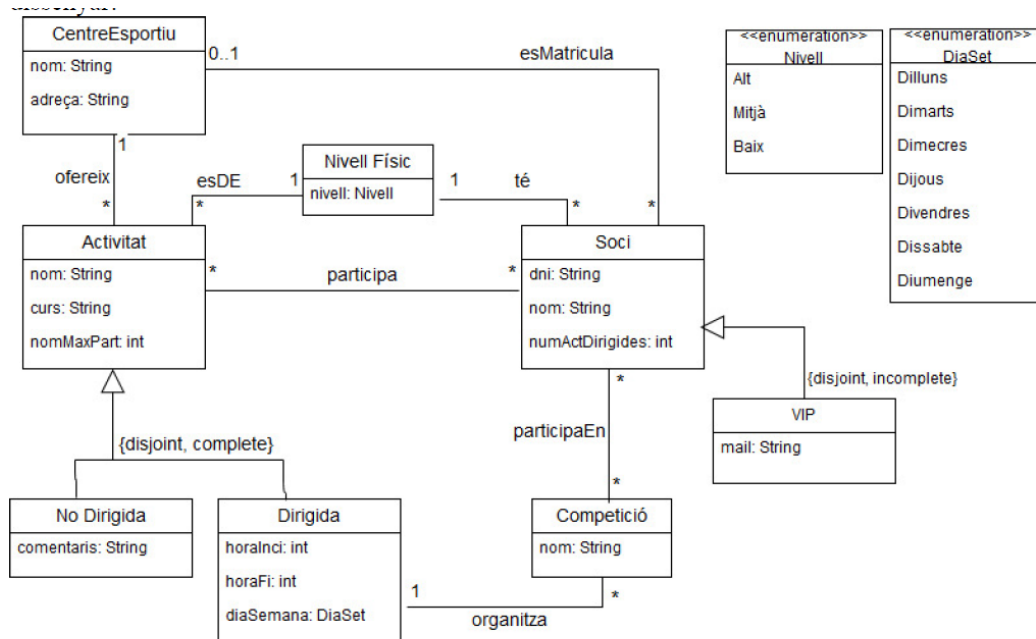
- Per comprovar que les matèries de la publicació són totes de la mateixa àrea es podria fer en una operació en el CercadorMateries
 - o `comprovarArea(setMateries: set(codiM:String)):Booleà`
 - o Retorna cert si tots els codis de matèria són del mateixa àrea, i fals en cas contrari.
 - o La sentència SQL que es podria usar podria ser similar a:

```
select count(distinct m.codiA)
from materies m
where m.codiM in setMateries
```
 - o Si el número que retorna és 1 aleshores el booleà ha de ser cert, si és superior a 1 ha de ser fals.
 - o De totes maneres cal igual un primer bucle per inserir les files a la taula MateriesPublicacio
- Per buscar les dades de les biblioteques es podria fer una operació en el CercadorBiblioteques
 - o `obtenirNumPrestecsEnCurs():set(String, String, Int)`
 - o La sentència SQL que es podria usar podria ser similar a:

```
select b.codiB, b.nomB, count(*)
from biblioteques b, exemplars e, prestecs p
where b.codiB=e.codiB and
      e.codiP=p.codiP and
      e.codiE=p.codiE and
      p.dataTornat is null
group by b.codiB
```
 - o Invocant aquesta operació sobre el cercador ja n'hi ha prou, no cal fer res més.

Exercici 9

Un organisme que organitza activitats recreatives en centres esportius vol dissenyar un sistema software per enregistrar els centres, les activitats, els socis i les competicions en les que hi participen. A continuació disposeu d'un fragment de l'esquema conceptual i del contracte d'una operació del sistema a dissenyar:



R.I. Textuals:

- Claus: (CentreEsportiu, nom); (NivellFísic, nivell); (Soci, dni); (Competició, nom)
- Un centre esportiu no pot tenir dues activitats amb el mateix nom, però centres diferents si.
- Un soci d'un nivell físic només pot participar en activitats del mateix nivell físic.
- Un soci només pot participar en activitats del centre esportiu on està matriculat
- Un centre esportiu pot oferir com a màxim 30 activitats en el mateix curs.
- Una activitat dirigida pot tenir com a màxim 4 socis no VIP
- L'hora d'inici i final són al rang 1.. 24
- L'hora d'inici d'una activitat dirigida és menor que l'hora de fi
- Un soci que no sigui VIP pot participar com a màxim a dues activitats dirigides.
- Les competicions en les que participa un soci són competicions organitzades per les activitats dirigides en les que participa.
- En una activitat participen com a màxim el numero de socis denotat per l'atribut numMaxPart de l'activitat
- Tots el atributs de tipus integer i float han de ser positius
- Altres restriccions no rellevants pel problema ...

context CapaDomini:: AltaSociVIP (dni: String, nom: String, nomCentreEsp: String, nomsActivitats: Set(nomAct: String), nomNivell: Nivell, eMail: String): Set (nomComp: String)

pre centreEsportiuExisteix: el centre esportiu amb nom *nomCentreEsp* existeix.

pre activitatExisteix: les activitats del centre *nomCentreEsp* amb nom *nomAct* existeixen.

pre nivellFísicExisteix: el nivell físic *nomNivell* existeix.

exc sociExisteix: el soci amb el dni *dni* ja existeix.

exc numMaxPartExcedit: alguna de les activitats *nomCentreEsp*, *nomAct* superen el *numMaxPart* en afegir el nou soci com a participant.

exc diferentNivell: alguna de les activitats *nomCentreEsp*, *nomAct* tenen un nivell diferent de *nomNivell*.

post altaSoci: s'ha creat el Soci VIP creant les associacions necessàries amb classes *Nivell*, *CentreEsportiu* i amb les activitats en les que participarà el soci de la classe *Activitat*.

post establirNumActivitatsDirigides: el valor *numActDirigides* del nou soci correspon al número d'activitats dirigides en les que participa.

post result = nom de les competicions on es pot apuntar per a participar el nou Soci (totes les competicions d'activitats dirigides en les que el soci participa).

Es demana:

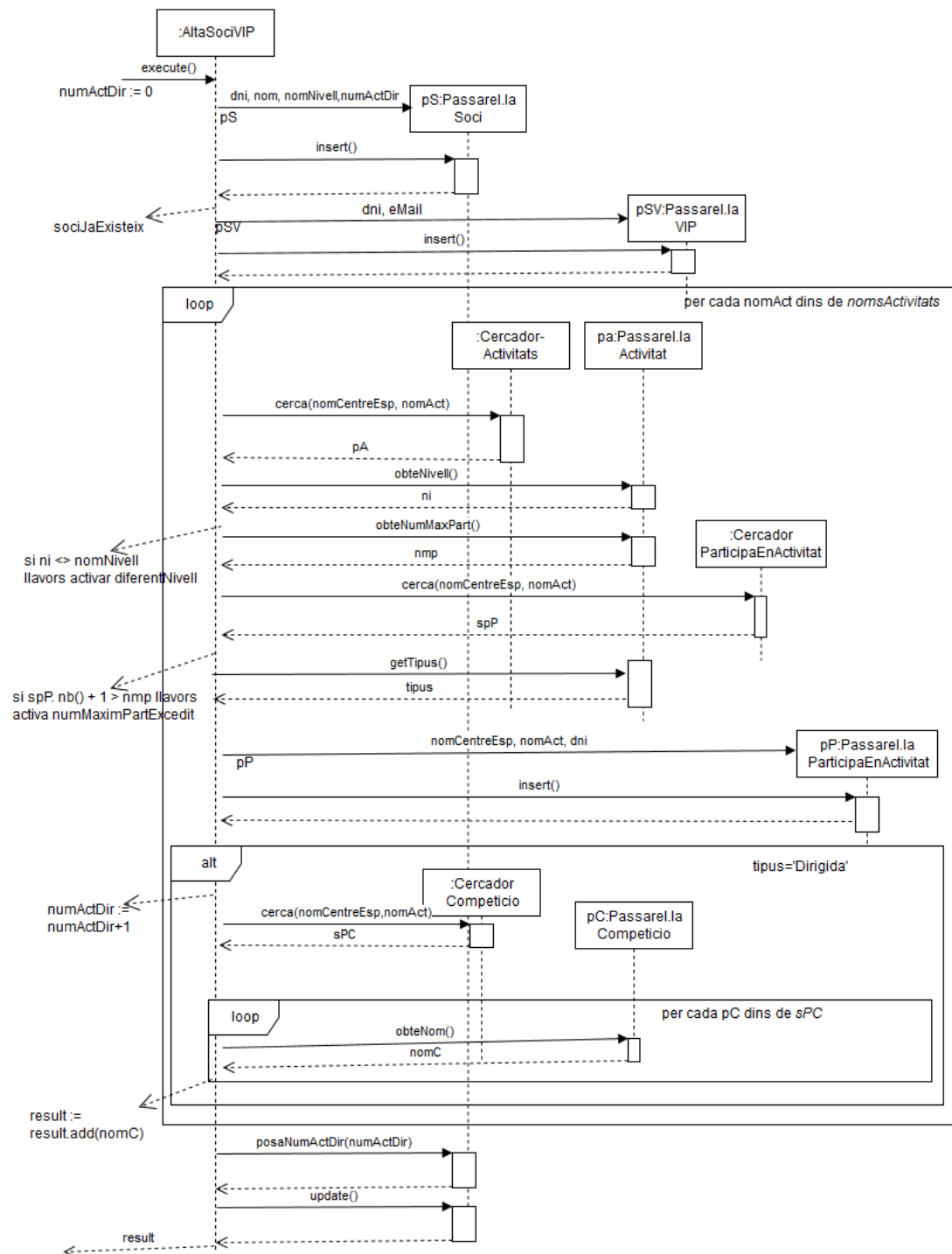
- 1) Esquema de la base de dades usant *Class Table Inheritance* per a les jerarquies. Es podeu usar restriccions de claus primària, forana i altres restriccions de columna i/o taula que poseu a la base de dades (no podeu usar triggers).
- 2) Feu el disseny de l'operació *AltaSociVIP* usant patrons *Transaction Script amb Row Data Gateway* (Pasarel.la Fila) (*sense operacions arbitràries de consulta*). Indiqueu clarament quines restriccions que heu definit en el disseny de la base de dades useu en el disseny d'aquesta operació.
- 3) Expliqueu en cas de poder definir triggers i/o operacions arbitràries de consulta quins canvis introduiríeu en el disseny de la transacció anterior.
 - a. En cas d'afegir un trigger cal que indiqueu quin esdeveniment l'activaria (insert/delete/update) i sobre quina taula. També cal explicar què es faria dins del procediment executat en activar-se el trigger.
 - b. En cas d'usar una o més operacions arbitràries de consulta, cal donar per cada operació, el nom de l'operació, els paràmetres d'entrada, els paràmetres de sortida i la/es sentència/es SQL que s'executaria dins de l'operació.

Solució

1)
centreEsportiu(nomCentreEsp, adreça)
nivellFisic(nivell)
soci(dni, nom, numActDirigides, nivell, nomCentreEsp)
 {nivell} referencia nivellFisic NOT NULL
 {nomCentreEsp} referencia centreEsportiu
vip(dni, eMail)
 {dni} referencia soci
activitat(nomCentreEsp, nomAct, curs, numMaxPart, nivell, tipus)
 {nivell} referencia nivellFisic
 {nomCentreEsp} referencia centreEsportiu
activitatNoDirigida(nomCentreEsp, nomAct, comments)
 {nomCentreEsp, nomAct} referencia activitat
activitatDirigida(nomCentreEsp, nomAct, horaInici, horaFi, diaSetmana)
 {nomCentreEsp, nomAct} referencia activitat
participaEnActivitat(dni, nomCentreEsp, nomAct)
 {dni} referencia soci
 {nomCentreEsp, nomAct} referencia activitat
Competicio(nomComp, nomCentreEsp, nomAct)
 {nomCentreEsp, nomAct} referencia activitatDirigida NOT NULL
participaEnCompeticio(nomComp, dni)

{nomComp} referencia competicio
 {dni} referencia soci

2)



3 a)

Es pot fer un trigger que controli les excepcions diferentNivell i numMarPartExcedit.

- Esdeveniment que l'activa
 - o Insert, taula participaEnActivat
 - o En fer l'insert de la participació es mirarà que el número de participacions més 1 sigui inferior que numMaxPart.

o També es mirará que el nivell de l'activitat és el mateix que la del soci.

3 b)

Es pot fer que el CercadorCompeticio tingui una operació per retornar el conjunt de strings que són els noms de competicions d'una certa activitat dirigida

- `obteNomsCompeticions(nomCE: String, nomA:String): Set(nomC: String)`.

Sentència SQL

Select nomC

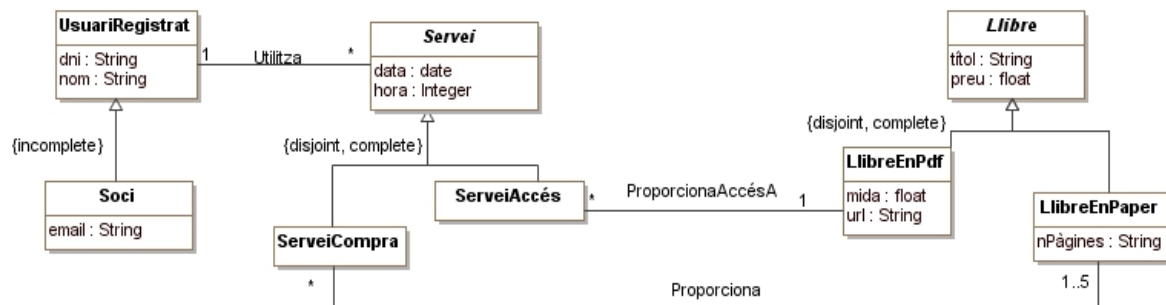
from competicions

Where nomCentreEsp = nomCE and nomAct = nomA

O bé una única operació que donat el conjunt d'activitats a les que s'apunta el soci doni ja tots els noms de competicions.

Exercici 10

Una llibreria que ven llibres per internet vol dissenyar un sistema software per enregistrar els llibres que ven als seus clients. A continuació disposeu d'un fragment l'esquema conceptual del sistema a dissenyar:



R.I. Textuals:

- Claus: (UsuariRegistrat, dni); (Soci, email); (Servei, UsuariRegistrat:dni + data + hora); (Llibre, títol); (LlibreEnPdf, url)
- Un usuari registrat que no sigui soci només pot comprar com a mínim un i com a màxim tres llibres en paper a cada servei de compra.
- Un usuari registrat que no sigui soci només pot accedir un únic cop a un mateix llibre en pdf.
- Un llibre en pdf es pot accedir com a molt 100 vegades en una data.
- Un llibre pot ser comprat com a molt 3 vegades (en serveis de compres diferents) per un soci.

Considereu l'operació de la capa de domini:

context CapaDomini:: *EsborrarServeisAntics* (data: date): Set (String)

pre dataCorrecta: la data data és vàlida i no és futura.

exc noHiHaServeis: no hi ha serveis al sistema.

exc noHiHaServeisAntics: hi ha serveis al sistema però no hi ha serveis amb data anterior a la data indicada al paràmetre.

post esborrarServeisAntics: s'eliminen tots els serveis amb data anterior a la data indicada al paràmetre.

post result= dni dels socis que s'han quedat sense cap servei assignat.

Es demana:

- a) Esquema de la base de dades usant *Concrete Table Inheritance* per a la jerarquia de Llibre i *Class Table Inheritance* per a les jerarquies de UsuariRegistrat i Servei. Les responsabilitats que es poden assignar a l'esquema de la base de dades són les de clau primària, clau forana i restricció UNIQUE.

- b) Feu el disseny de l'operació *EsborrarServeisAntics* usant el patró *Transaction Script* amb *passarel·la* fila (Row Data Gateway) sense operacions arbitràries de consulta.

Solució

a)

UsuariRegistrat (dni, nom)

Soci (dni, email)

{dni} referencia UsuariRegistrat

email UNIQUE

Servei (dni, data, hora, tipus)

{dni} referencia UsuariRegistrat

ServeiCompra (dni, data, hora)

{dni, data, hora} referencia Servei

ServeiAccés (dni, data, hora, títol)

{dni, data, hora} referencia Servei

{títol} referencia LlibreEnPdf

LlibreEnPdf (títol, preu, mida, url)

url UNIQUE

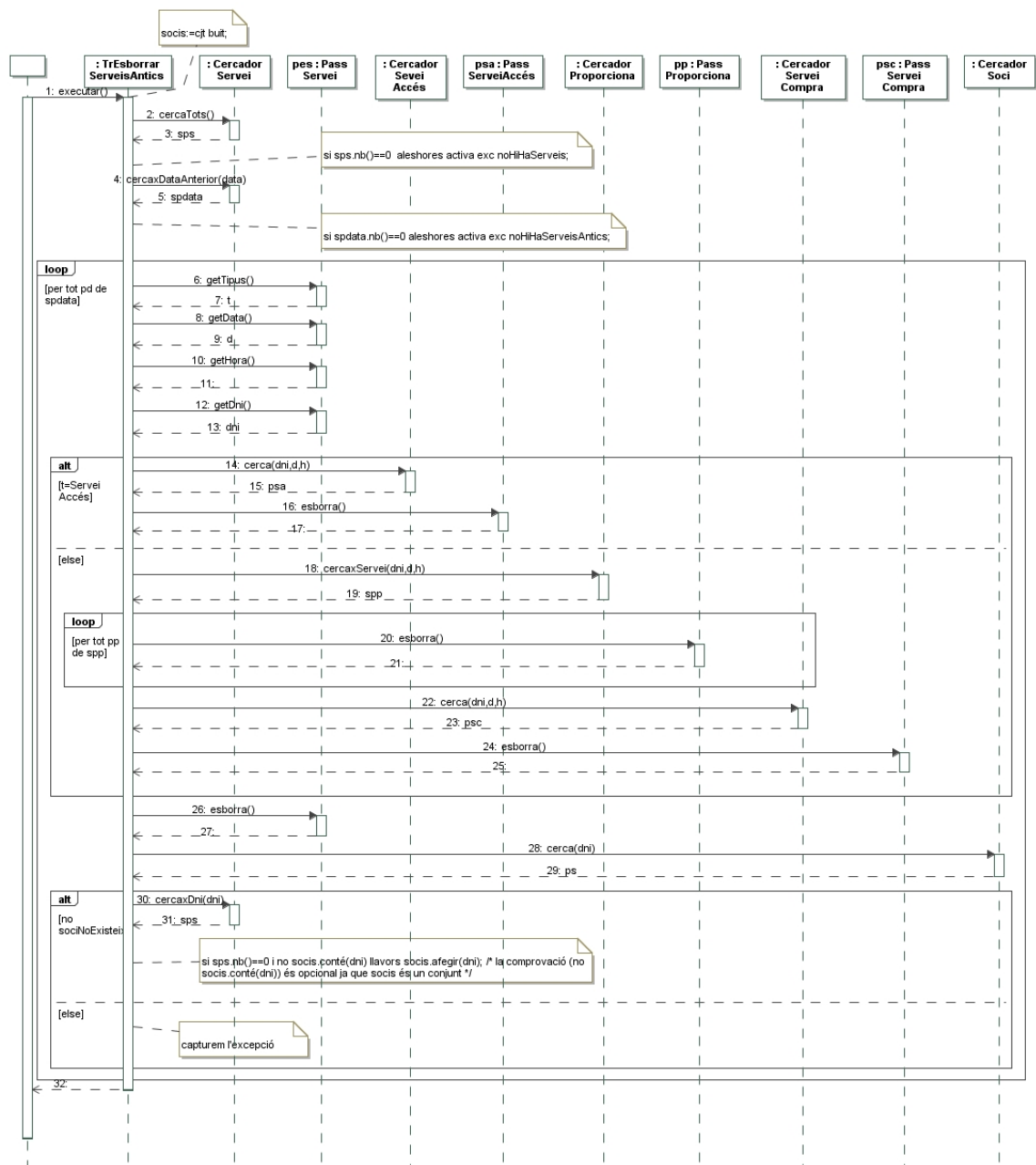
LlibreEnPaper (títol, preu, nPàgines)

Proporciona (dni, data, hora, títol)

{dni, data, hora} referencia ServeiCompra

{títol} referencia LlibreEnPaper

b)



Hem obviat la destrucció de passarel·les.